

Oraxle Skill — Demo Script

Target: 4 min screen recording for DoraHacks submission

Pre-recording setup (do once, off-camera)

```
# 1. Install the skill into Claude Code
npx skills add https://github.com/7maylord/oraxle -g
# (or: cp -r ~/.claude/skills/oraxle if working from local clone)

# 2. Install dependencies
cd ~/.claude/skills/oraxle && pnpm install

# 4. Open a fresh Claude Code session in a clean directory
cd ~/Desktop && claude
```

Recording checklist before you hit record

- ☐ Terminal font size 18+ (readable on 1080p)
 - ☐ Clean terminal, no prior output visible
 - ☐ `.env` has `PHAROS_RPC_URL` set
 - ☐ Internet connection live (fetching from Pharos Atlantic)
 - ☐ Clock hidden or paused (prices change each run — that's the point)
-

Scene-by-scene

INTRO — 20 seconds

What to show: repo structure + SKILL.md open side-by-side

Say:

"Pharos is built for real-world assets. Agents operating on Pharos — sizing payments, triggering liquidations, rebalancing portfolios — need current asset prices. Oraxle is a Claude Code skill that gives any agent verified Chainlink prices from Pharos Atlantic Testnet with one line."

Type in terminal:

```
ls ~/.claude/skills/oraxle-skill
```

SCENE 1 — `list_feeds` — 30 seconds

Type this prompt in Claude Code:

What price feeds are available on Pharos right now?

Expected Claude response (routes to list_feeds):

```
[
  { "key": "BTC/USD", "heartbeat": 3600, "active": true },
  { "key": "ETH/USD", "heartbeat": 3600, "active": true },
  { "key": "PROS/USD", "heartbeat": 3600, "active": true },
  { "key": "LINK/USD", "heartbeat": 3600, "active": true },
  { "key": "USDT/USD", "heartbeat": 3600, "active": true },
  { "key": "USDC/USD", "heartbeat": 3600, "active": true },
  { "key": "SOL/USD", "heartbeat": 3600, "active": true },
  { "key": "XRP/USD", "heartbeat": 3600, "active": true },
  { "key": "BNB/USD", "heartbeat": 3600, "active": true }
]
```

Say:

"Nine feeds — including PROS, the native Pharos token. The agent discovered this itself. No hardcoded addresses, no config files."

SCENE 2 — get_staleness_report — 30 seconds

Type this prompt:

Is the ETH/USD oracle fresh enough to use before I make a trade?

Expected output:

```
{
  "assetKey": "ETH/USD",
  "status": "live",
  "isStale": false,
  "ageSeconds": 87,
  "maxAgeSeconds": 5400,
  "updatedAt": 1781298974,
  "updatedAtIso": "2026-06-12T21:16:14.000Z"
}
```

Say:

"Before committing to any trade the agent checks the oracle first — not the price, just whether the data is safe to act on. Live, 87 seconds old, max allowed 5,400. Green light to proceed."

SCENE 3 — get_price — 30 seconds

Type this prompt:

Get me the current BTC price

Expected output:

```
{
  "assetKey": "BTC/USD",
  "price": "63389.66015689818",
  "updatedAt": 1781297360,
  "updatedAtIso": "2026-06-12T20:49:20.000Z",
  "isStale": false,
  "source": "chainlink-pharos-atlantic"
}
```

Say:

"Real BTC price from Chainlink Push Engine on Pharos Atlantic Testnet — 18-decimal precision, staleness validated, source tagged. The agent didn't write any RPC code. It called one tool."

SCENE 4 — `get_multi_price` — 30 seconds

Type this prompt:

I need to rebalance a portfolio. Get me prices for BTC, ETH, and PROS all at once.

Expected output:

```
[
  {
    "assetKey": "BTC/USD",
    "price": "63389.66015689818",
    "isStale": false,
    "source": "chainlink-pharos-atlantic"
  },
  {
    "assetKey": "ETH/USD",
    "price": "1666.82870585851945",
    "isStale": false,
    "source": "chainlink-pharos-atlantic"
  },
  {
    "assetKey": "PROS/USD",
    "price": "0.609429135",
    "isStale": false,
    "source": "chainlink-pharos-atlantic"
  }
]
```

Say:

"Three feeds fetched in parallel. Notice PROS/USD — the Pharos native token at \$0.609, live from the chain. A portfolio agent can now compute total value and decide whether to rebalance."

SCENE 5 — `get_price_history` — 30 seconds

Type this prompt:

Show me the last 3 ETH price rounds – I want to know if it's been trending down before I decide to liquidate.

Expected output:

```
{
  "assetKey": "ETH/USD",
  "rounds": 3,
  "history": [
    { "price": "1666.82870585851945", "updatedAtIso": "2026-06-12T21:16:14.000Z" },
    { "price": "1671.24510000000000", "updatedAtIso": "2026-06-12T20:13:58.000Z" },
    { "price": "1678.64392500000000", "updatedAtIso": "2026-06-12T19:09:44.000Z" }
  ],
  "source": "chainlink-pharos-atlantic"
}
```

(Note: if history is empty on testnet, say: "The Push Engine on testnet doesn't emit AnswerUpdated events yet — on mainnet this returns full history. The tool handles it gracefully and returns an empty array rather than erroring.")

Say:

"Price history pulled from on-chain events. \$1,678 → \$1,671 → \$1,666 — three rounds, trending down. A liquidation agent now has the context it needs to act with confidence, not just a single data point."

SCENE 6 — `get_depeg_alert` — 30 seconds

Type this prompt:

Before I add liquidity to the USDC pool, check if USDC is depegged right now.

Expected output:

```
{
  "assetKey": "USDC/USD",
  "status": "healthy",
  "price": "0.9997337",
  "pegTarget": "1",
  "deviationBps": 3,
  "deviationPct": "0.0266%",
  "updatedAt": 1781299534,
  "updatedAtIso": "2026-06-12T21:25:34.000Z"
}
```

Say:

"Healthy — 3 basis points off peg, well within the 50 bps warning threshold. Safe to deposit. If USDC were at \$0.96 right now, this would return 'critical' and the agent would abort the deposit automatically. This is exactly the guardrail Pharos liquidity pools need."

SCENE 7 — compare_prices — 30 seconds

Type this prompt:

How much ETH is 1 BTC worth right now? I need to size a cross-asset payment.

Expected output:

```
{
  "baseKey": "BTC/USD",
  "quoteKey": "ETH/USD",
  "ratio": "38.030098",
  "meaning": "1 BTC = 38.0301 ETH",
  "basePrice": "63389.66015689818",
  "quotePrice": "1666.82870585851945",
  "updatedAt": 1781297360,
  "updatedAtIso": "2026-06-12T20:49:20.000Z"
}
```

Say:

"1 BTC equals 38.03 ETH — computed with BigInt arithmetic on the raw 18-decimal values, no floating-point drift. A payment agent can multiply this ratio by the transfer amount and get an exact figure safe to send on-chain."

OUTRO — 30 seconds

What to show: SKILL.md open, Capability Index table visible

Say:

"Seven tools. One skill. Any agent on Pharos can add verified RWA price data to its decision loop with a single skill invocation — no contracts to deploy, no wallets required, read-only calls to Chainlink's Push Engine running live on Pharos Atlantic Testnet right now.

This is the oracle layer every Pharos DeFi agent needs."

Show the Capability Index table in SKILL.md as final frame.

Timing summary

Scene	Tool	Time
Intro	—	0:00 – 0:20
1	list_feeds	0:20 – 0:50
2	get_staleness_report	0:50 – 1:20
3	get_price	1:20 – 1:50
4	get_multi_price	1:50 – 2:20

5	get_price_history	2:20 – 2:50
6	get_depeg_alert	2:50 – 3:20
7	compare_prices	3:20 – 3:50
Outro	—	3:50 – 4:20

Total: ~4:20 — safely under the 5-minute limit.

If Claude Code doesn't activate the skill automatically

Install the skill manually and confirm it loaded:

```
# In Claude Code terminal
/skills
# Should show: oraxle-skill
```

If still not activating, prepend to your prompt:

```
Using the oraxle-skill: [your prompt here]
```

DoraHacks submission description (copy-paste ready)

Oraxle – Verified Chainlink Price Feeds for Pharos Agents

Pharos is built for tokenized real-world assets, but agents have no verified price source to act on. Oraxle fills this gap.

A composable Agent Skill that exposes seven tools backed by Chainlink's Push Engine running natively on Pharos Atlantic Testnet (chain ID 688689):

```
get_price           – single asset price, staleness validated
get_multi_price      – batch pricing for portfolio decisions
get_staleness_report – oracle health pre-flight check
list_feeds           – dynamic feed discovery (9 assets)
get_price_history    – trend data via on-chain AnswerUpdated events
get_depeg_alert      – stablecoin peg monitor with configurable thresholds
compare_prices       – cross-asset ratio, BigInt arithmetic, no float drift
```

Every price is validated before being returned. Supports BTC, ETH, PROS, LINK, USDT, USDC, SOL, XRP, BNB – the assets that matter for Pharos DeFi.

Any agent on Pharos can add verified price data to its decision loop with a single skill invocation.

Stack: TypeScript · MCP SDK · Ethers.js · Chainlink Push Engine · Zod · Vitest · Pharos Atlantic Testnet